A Didactic Dissection of Data Structures

Workshop – Term 1, Week 5

Susan He, Austin Song

Outline

1 Housekeeping

2 Ranges and pointers

3 What is a data structure?

4 A range sum thing?

5 Never heard of a linked list

6 AVL trees in practice

7 COMBO

8 Appendix and resources

Attendance!

Please scan the attendance form so you can get free food

VJudge For Workshop

If you're confident with the content for today, feel free to get started on problems!

Link: https://vjudge.net/contest/797040

Password: `cpmsoc`

Outline

1 Housekeeping

2 Ranges and pointers

3 What is a data structure?

4 A range sum thing?

5 Never heard of a linked list

6 AVL trees in practice

7 COMBO

8 Appendix and resources

A Problem

Going Viral

The marketing team has accidentally granted you access to the CPMSoc Instagram account. You've been putting out banger reels once a day the past n days, with the ith reel accruing some number of likes, a_i, which may be negative.

To see if you have a shot at virality, the marketing team would like to find the highest average number of likes over any k day period, where $1 \leq k \leq n$. Note, k is fixed and given to you. Since they don't program, but you do, they'd like you to do this for them.

Going Viral

First observation:

- For an interval starting on day i, the average is $\frac{1}{k}(a_i + a_{i+1} + \cdots + a_{i+k-1})$.
- Since the $\frac{1}{k}$ is the same for any period, we can just find the maximum *sum* over any period of k days, then divide the answer by k.

This gives us a straightforward naive solution.

Solution 1

For each start day $1 \leq i \leq n+1-k$, consider each day $j \in [i, i+k-1]$, and add a_j to a total for the period, s_i. Then, take the max s_i, divide it by k, and return the result.

Time Complexity

- For each of $O(n-k)$ start points, we add k numbers, giving $O(k(n-k))$.
- In the worst case if $k = \frac{n}{2}$, this is $O(n^2)$.

Going Viral - Can We Do Better?

Second observation:

- The k day periods for consecutive start days are very similar!
- E.g. even though the periods $[100, 1000]$ and $[101, 1001]$ are big, the only difference is that day 1001 has been added to the end, and day 100 has been removed from the front.
- Could we share some information between these two intervals?

Sliding Window

A 'window' here refers to some range $[l, r]$ of an array a. Suppose we would like to know some property $P(l, r)$ of the range. If we are able to perform the following:

- Determine $P(l, r+1)$ from $P(l, r)$ and $a[r+1]$,
- Determine $P(l+1, r)$ from $P(l, r)$ and $a[l]$,

in $O(1)$, then we can 'slide' the window from $[l, r]$ to $[l+1, r+1]$ while maintaining the property in $O(1)$.

Going Viral - We CAN Do Better!

We use this technique in our modified solution:

Solution 2

First, for $i = 1$, we find the property $s(1, k)$, by taking the sum of the first k elements. Then, for each $2 \leq i \leq n + 1 - k$, we take the old sum $s(i - 1, i + k - 2)$, and 'slide the window' over to find $s(i, i + k - 1)$:

- $s(i - 1, i + k - 1) = s(i - 1, i + k - 2) + a[i + k - 1]$
- $s(i, i + k - 1) = s(i - 1, i + k - 1) - a[i - 1]$

We then similarly take the max $s(i, i + k - 1)$, divide it by k, and return the result.

Time Complexity

- We sum the first k elements in $O(k)$, then perform $n - k$ slides, each in $O(1)$.
- This means the overall complexity is $O(k + n - k) = O(n)$ yay!

A New Problem

Some Eggcellent Sums

You are at the grocery store and would like to buy some eggs. There are n eggs in a line, where the ith egg contains x_i yolks within, where $1 \leq x_i \leq 1e6$, which is written on the outside of the egg. You would like to prepare a feast that requires exactly k egg yolks.

To acquire your eggs, you ask Austin to go to egg number i and ask Matthew to go to egg number j, where $j \geq i$. The two then run towards each other, grabbing all the eggs along the way (including at their starting positions). How many different pairs (i, j) will result in you obtaining k eggs?

Constraints

- $2 \leq n \leq 1e6$
- $1 \leq x_i \leq 1e6$
- $1 \leq k \leq 1e12$

Some Eggcellent Sums - Some Solutions

Solution 1

For every $1 \leq i < j \leq n$, iterate over the subarray and find the sum, checking if it is equal to k. There are $O(n^2)$ different subarrays, and it takes $O(n)$ to find the sum of the subarray, so this solution is $O(n^3)$, which isn't particularly good.

Solution 2

For every $1 \leq i < n$, iterate over all j from i to n, keeping a running sum as you go. We still consider $O(n^2)$ different subarrays, but this time the sum is updated in $O(1)$ on each step, so the algorithm is $O(n^2)$, which is better but still too slow.

Some Eggcellent Sums - Observation Time!

- Suppose we found some solution (i, j).
- Suppose there was also some solution $(i + 1, j')$.
- What can we say about j and j'?
- We have to have $j' > j$!

This will prove very useful for the below idea!

The Two Pointer Algorithm

Suppose that we are trying to find some subarrays of a that meet a certain property. If we can show that for any 2 valid subarrays $a[i...j]$ and $a[i'...j']$ with $i \leq i'$ that $j \leq j'$, we can use the two pointer algorithm:

- Set the left (i) right (j) pointers to 1 initially.
- While the left pointer i is less than n:
  - While there could still be a solution using a bigger j, increment j.
  - Increment i.

Some Eggcellent Sums - Two Pointer Solution

Solution 3

- Set i, j to 1 initially.
- While $i \leq n$:
  - While the current sum from i to j is less than k, increment j and update the running sum (don't put j past n tho).
  - Once the sum is at least k (or $j = n$), add one count to our answer if the sum is exactly k.
  - Increment i, updating the running sum by subtracting the old x_i.
- Observe that we never miss any solutions due to the two pointer property holding.
- **Complexity:** Notice that both i and j are incremented at most n times, since they never move backwards. All other checks are $O(1)$. Hence, the overall algorithm is $O(n)$, which fits in time!

No Longer Viral :(

Problem

After failing to go viral online, you've decided to make friends the old fashioned way. For each of the past n days, you've spoken to one person, numbered x_i. Note, you can speak to the same person on different days.

You would like to find the largest number of consecutive days in which you did not speak to the same person more than once.

Constraints:

- $1 \leq n \leq 1e6$
- $1 \leq x_i \leq n$
- $1 \leq k \leq n$

No Longer Viral :(- Ideas!

Thoughts

- One natural idea is to consider each possible start point.
- So, we could try to find for each i, the largest j such that $x_i, ..., x_j$ are all distinct. We would then take the largest range we find.
- Observation: If the furthest endpoint for start day i is j, what can we say about the furthest endpoint for start day $i + 1$?
- It has to be at least as far, since $[i + 1, j]$ is a subarray of $[i, j]$, so if all are distinct in $[i, j]$, then all are distinct in $[i + 1, j]$.
- This means we can use a 2 pointer idea!

No Longer Viral - Two Pointer Solution

- Set i, j to 1 initially.
- While $i \leq n$:
  - While the x_{j+1} is not already in $x_i, ...x_j$, increment j (don't put j past n tho).
  - Record the size of the interval, being $j - i + 1$.
  - Increment i.
- We then take the largest interval size found.
- Question: How do we check if x_{j+1} is already found?
  - Can keep a 'seen' array $seen[1...n]$, mark a friend as 'seen' once they enter the interval, and mark them unseen when they leave.
  - Bonus: What if x_i can go up to $1e9$?
- **Complexity:** Both i and j are incremented at most n times, since they never move backwards. All other checks are $O(1)$. Hence, the overall algorithm is $O(n)$, which is good!

Outline

1 Housekeeping

2 Ranges and pointers

3 **What is a data structure?**

4 A range sum thing?

5 Never heard of a linked list

6 AVL trees in practice

7 COMBO

8 Appendix and resources

The Language of Data Structures

Data structures are objects that store data and perform actions on said data.

Queries and Updates

In competitive programming, these actions (**operations**) often come in two forms:

1. **Queries:**
   - We ask something about the data (e.g. "what's the sum of the values stored?").
   - The DS extracts this information and hands this to us.
2. **Updates:**
   - We ask to modify the data (e.g. "add 1 to all the values stored here").
   - The DS updates its data to match the request.

Data structures are almost solely categorised by the speed at which and method by which they perform these operations.

Examples

- What's a data structure which takes N ordered integers and supports the following?
  - Set the i'th integer to x in $O(1)$.
  - Query the value of the i'th integer in $O(1)$.
- What's a data structure which supports the following?
  - Add an integer x to a list in $O(\log N)$.
  - Query the greatest value in the list in $O(1)$.
  - Remove the greatest value in the list in $O(\log N)$.

...and here is an example of what a data structure can do!

- Query the sum of a range in $O(\log N)$.
- Insert an integer array A after the i'th value in $O(\log N)$.
- Reverse any subarray $[l, r]$ in $O(\log N)$.
- Add an arithmetic sequence to a range in $O(\log N)$.

Outline

1 Housekeeping

2 Ranges and pointers

3 What is a data structure?

4 **A range sum thing?**

5 Never heard of a linked list

6 AVL trees in practice

7 COMBO

8 Appendix and resources

A Problem

Rob the Race-ist

Rob is a race-ist – he likes running races. A race consists of N sections, with the ith section being d_i kilometers long. Amateur race-ists like Rob do not need to run the whole race, just some continuous range $[l, r]$, which spans all sections from l to r (inclusive).

Unfortunately, Rob is also a racist. He keeps changing his mind about which sections he wants to run to avoid *certain* types of people. He has Q questions, with the ith question asking for the total distance of sections $[a_i, b_i]$. Help Rob answer his questions.

Part I: Try solving this in $O(NQ)$.

Part II: Try solving this in $O(N + Q)$.

Rob the Race-ist – Just Do It

Brute Force

For each of Rob's Q questions, we simply sum up the distance of the sections in that range. Each question i specifies a range of sections $[a_i, b_i]$, so we can sum up the values of $[d_{a_i}, d_{b_i}]$ using a for-loop.

Time Complexity

- In the worst case, each of Rob's questions asks for a range of size $O(N)$.
- Since the loop is $O(N)$ for each question, it is $O(NQ)$ in total.

Rob the Race-ist – Quicker Queries

- We want to answer queries more quickly by doing some *precomputation* beforehand.
- Think about how the range below could be composed from two other ranges...

3	3	4	5	2	2	4	3

3	3	4	5	2	2	4	3

3	3	4	5	2	2	4	3

Rob the Race-ist – Quicker Queries

Prefix Sums

Every range $[l, r]$ of an array can be composed as the difference between the ranges $[1, r]$ and $[1, l-1]$. That is,

$$\sum_{i=l}^{r} a_i = \sum_{i=1}^{r} a_i - \sum_{i=1}^{l-1} a_i$$

We can precompute an array p, with $p_i = \sum_{j=1}^{i} a_j$. Because then, any range sum can be expressed as a difference of two p_i values:

$$\sum_{i=l}^{r} a_i = p_r - p_{l-1}$$

Rob the Race-ist - Quicker Queries

Prefix Sums

We can precompute the array p in one for-loop, using the formula $p_i = p_{i-1} + d_i$. Then, answering Rob's questions happens in constant time.

Time Complexity

- Precomputing the array p is $O(N)$.
- It takes $O(1)$ time to answer each of Rob's Q questions.
- In total, it is $O(N + Q)$.

Another Problem

Subsequences Summing to Sevens

Farmer John has N cows, who are standing in a line. Each cow i has a distinct integer ID number a_i. Farmer John would like to take a photo of a contiguous group of cows but, due to a traumatic childhood incident involving the numbers 1...6, he only wants to take a picture of a group of cows $[l, r]$ if their IDs add up to a multiple of 7.

Please help Farmer John determine the size of the largest group that can be in the photograph. He won't accept anything worse than $O(N)$ time complexity.

Subsequences Summing to Sevens – Samples

Sample Cases

Sample Input:

```
7
3 5 1 6 2 14 10
```

Sample Output:

```
5
```

Subsequences Summing to Sevens – Reformulate

Let's reformulate the condition for what makes a valid group of cows.

We can take a group of cows $[l, r]$ if and only if

$$\sum_{i=l}^{r} a_i \equiv 0 \pmod 7$$

$$p_r - p_{l-1} \equiv 0 \pmod 7$$

$$p_r \equiv p_{l-1} \pmod 7$$

the prefix sum value of $l - 1$ and r are congruent mod 7.

Subsequences Summing to Sevens – Reformulate

Prefix Sums

Note that any p_i can only take on values $0 \ldots 6 \pmod 7$. So for every possible value $x \in \{0, 1, \ldots, 6\}$, we find the first and last index such that $p_i \equiv x \pmod 7$.

This gives us an answer for each x, since $|[l, r]| = r - l + 1 = r - (l - 1)$, i.e. the difference between these two furthest indices. Maxxing the answers across all x values gives the final answer.

Time Complexity

- Precomputing the array p is $O(N)$.
- It takes $O(N)$ time to find the first and last indices for each x.
- In total, it is $O(N)$.

Outline

1 Housekeeping

2 Ranges and pointers

3 What is a data structure?

4 A range sum thing?

5 Never heard of a linked list

6 AVL trees in practice

7 COMBO

8 Appendix and resources

Arrays

Quick quiz on arrays!

- What's the time complexity of adding an element to the **end** of an array?
  - $O(1)$.
  - We call this operation `push_back`.
- What's the time complexity of deleting the **last** element of an array?
  - $O(1)$.
  - We call this operation `pop_back`.
- What's the time complexity of adding an element to the **start** of an array?
  - $O(N)$.
  - We call this operation `push_front`.
- What's the time complexity of deleting the **first** element of an array?
  - $O(N)$.
  - We call this operation `pop_front`.

Linked Lists

I don't know what these are.

Abusing the C++ STL

Good news – I don't need to care!

- C++ has several built-in DSAs in its **standard library** (affectionately termed the STL).
- This means you'll never have to implement another linked list again!

If you are using `g++` as your compiler (highly recommended), then

```
#include<bits/stdc++.h>
using namespace std;
```

should import most of everything! Else, import your packages manually (like in C):

```
#include<iostream>
#include<vector> // dynamic-sized arrays :o
...
#include<set>
using namespace std;
```

Queues

A **queue** is a data structure which supports `pop_front` and `push_back` in $O(1)$.

It also allows you to peek at the 'front' value only in $O(1)$.

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    queue<int> q; // q = {}
    q.push(1); // q = {1}
    q.push(2); // q = {1, 2}
    q.push(3); // q = {1, 2, 3}
    int x = q.front(); // x = 1
    q.pop(); // q = {2, 3}
}
```

These are often called **FIFO** structures (first in, first out).

Stacks

A **stack** is a data structure which supports `pop_back` and `push_back` in $O(1)$.

These are basically nerfed **vectors** (standard dynamic-sized arrays), since they can only peek at the 'top' value. (Guess what they are referred as if queues are FIFO...)

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st; // st = {}
    st.push(1); // st = {1}
    st.push(2); // st = {1, 2}
    st.push(3); // st = {1, 2, 3}
    int x = st.top(); // x = 3
    st.pop(); // st = {1, 2}
}
```

There's Still a Problem!

Here's a question –

"Aren't linked lists able to support all four operations in $O(1)$?"

Introducing the **deque**! (**D**ouble **E**nded **Que**ue)

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    deque<int> dq; // dq = {}
    dq.push_back(1); // dq = {1}
    dq.push_front(2); // dq = {2, 1}
    int x = dq.front(), y = dq.back(); // x = 2, y = 1
    dq.pop_back(); // dq = {2}
    dq.push_back(3); dq.push_back(1); // dq = {2, 3, 1}
    printf("%d", dq[1]); // wtf???
    dq.pop_front(); // dq = {3, 1}
}
```

Bracket Sequences

Problem

A valid **bracket sequence** S can be defined as:

- (A) or $\{A\}$ or $[A]$, if A is a valid bracket sequence,
- AB, if A and B are valid bracket sequences,
- The empty string.

Given a bracket sequence, determine whether it is valid in $O(N)$.

Examples:

$S = (()[]\{\})[]$ is valid, while $S' = ([)]\{()\})$ is invalid.

Bracket Sequences – Solution

- Theoretically, you want to match from inside out, eliminating adjacent bracket pairs.
  - i.e. $([\{\}()]) \to ([()]) \to ([]) \to () \to \emptyset$, so it's valid.
  - or $([\{\})] \to ([)]$ can't be reduced, so it's not valid.
- How can we be sure we're eliminating the right things?
  - *Observation*: Each closing bracket can only match validly to at most **one** opening bracket.
  - This happens to be the nearest opening bracket of the same type!
- We can compute the nearest opening bracket by maintaining a stack.

Algorithm:

- When we encounter an opening bracket, we add it to a stack.
- When we encounter a closing bracket:
  - If the top of the stack is a matching open bracket, we can eliminate the pair.
  - Otherwise, we can't ever match this closing bracket.
- If the stack is empty at the end, S is valid.
- Else, there are unmatched open brackets remaining and thus S is not valid.

Monotonic Stack?

Sumo Wrestling

There are N sumo wrestlers, and the i'th one has strength a_i. They are thrown into a ring one-by-one. When a sumo wrestler enters the ring, it throws out all strictly weaker sumo wrestlers currently in the ring, before standing idle. They don't throw out any other sumo wrestlers, and stay in the ring unless thrown out by a future wrestler.

For each wrestler, compute how many people they throw out.

Subtasks and Constraints:

- Subtask 1: $N \leq 10^3$, $1 \leq a_i \leq 10^9$.
- Subtask 2: $N \leq 10^6$, $1 \leq a_i \leq 10^9$.

Sumo Wrestling – Initial Thoughts

- Let's see what happens when we simply simulate the process.
  - Maintain a list L of all sumo wrestlers currently in the ring.
  - For each sumo wrestler, count the number of weaker wrestlers in the list and remove them accordingly. Then, add the wrestler to the ring.
  - This process is $O(|L|)$, since we have to check all elements in L.
- The total complexity of this is $O(N^2)$, since at worst, $|L| \sim N$.
- This passes the first subtask! But clearly we can do better.
- We always have to go through each sumo wrestler, so our best bet at optimising is to improve the efficiency of the $O(|L|)$ loop-through-check part.

Sumo Wrestling – Monotonicity

Let's draw each wrestler's strength as a bar with height a_i. What do we notice?

So how many times do we *actually* have to check/remove elements over all iterations?

Sumo Wrestling – Solution

Our list L needs to support three operations in quick time:

1. Query the end value.
2. Delete a value off the end.
3. Add a value onto the end.

Furthermore, L always stores a *non-increasing* subsequence of strengths in order of index.

- Since L is a stack in 'sorted' order, we call L a **sorted stack** or **monotonic stack**.

What's the time complexity? Is it still $O(N^2)$?

- Each element gets check-removed at most once over all N iterations.
- Each iteration also checks an extra final element.
- The total number of checks and check/removals in the whole algorithm is $\sim O(N)$.

Like two-pointers, using a monotonic stack *amortizes* the solution to $O(N)$ total!

Outline

1 Housekeeping

2 Ranges and pointers

3 What is a data structure?

4 A range sum thing?

5 Never heard of a linked list

6 **AVL trees in practice**

7 COMBO

8 Appendix and resources

Self-Balancing BSTs

Binary Search Trees

A **binary search tree** is a tree where each node holds a **value** such that:

- Every node has at most 2 children.
- For a given node with value v,
  - Its left child has value $x \leq v$.
  - Its right child has value $y > v$.

When the worst-case depth of the tree is $O(\log N)$, we say the tree is **balanced**.

There are various data structures which maintain a balanced BST. We call these **self balancing BSTs** (or affectionately SBBSTs).

You may have already encountered one of these in COMP2521 – AVL trees!

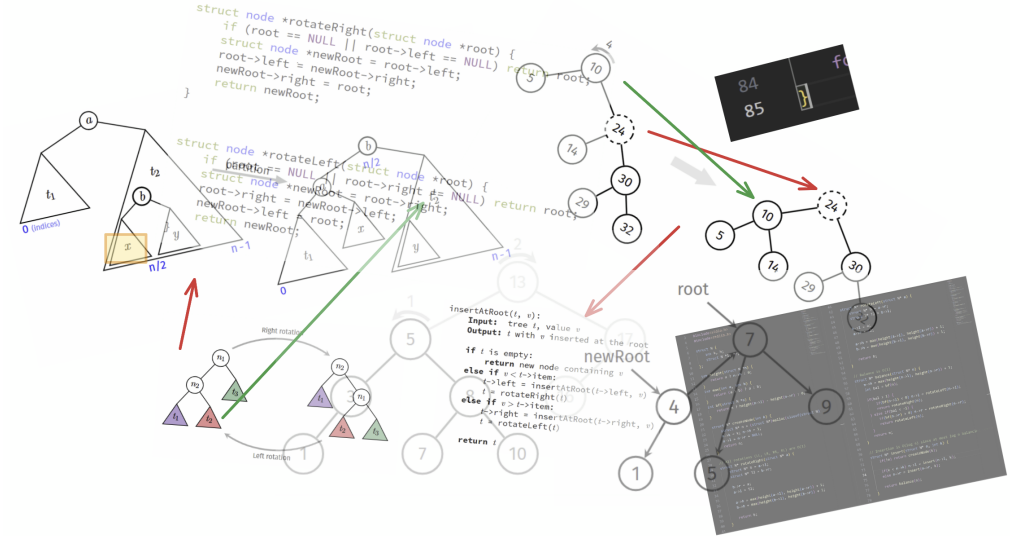
AVL Trees

These data structures allow:

- Add element x in $O(\log N)$,
- Query the minimum/maximum value in $O(1)$,
- Query the *order* of an element x (or confirm not present) in $O(\log N)$
- Delete element x in $O(\log N)$,

So they are quite useful for maintaining a sorted dynamic array! But how do we use one in a contest?

AVL Tree – Implementation



Sets

We can instead use C++'s standard built-in implementation!

A **set** maintains a sorted list of unique items, allowing for $O(1)$ query at start and end (min and max), and $O(\log N)$ insertions and deletions – it can do everything an AVL tree can.

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    set<int> s; // s = {}
    for(int i = 1; i <= 5; i += 2)
        s.insert(i);
    // s = {1, 3, 5}
    for(int i = 6; i >= 1; i--)
        s.insert(i);
    // s = {1, 2, 3, 4, 5, 6}
    printf("%d", *s.begin());
    s.erase(s.find(4)); // s = {1, 2, 3, 5, 6}
}
```

Maps

SBBSTs also let us store lookup tables – in C++ these are called **maps**.

They're basically just sets which store pairs instead of single elements:

- Add a (key: value) entry in $O(\log N)$,
- Remove a (key: value) entry in $O(\log N)$,
- Find the value stored at a key in $O(\log N)$,
- Edit the value stored at a key in $O(\log N)$.

Like a set, a map stores (key: value) pairs in sorted order (by key). To access a value,

```
set<pair<int, int>> s = {{0, 1}, {1, 2}, {2, 1}, {3, 2}};
map<int, int> mp = {{0, 1}, {1, 2}, {2, 1}, {3, 2}};

printf("%d", s.lower_bound({2, 0})->second); // very annoying
printf("%d", mp[2]); // very trivial :o
```

Multisets and Multimaps!

If you want to support duplicate values in your set, you can use a **multiset**!

For a multiset, the only notable functional differences are:

- You can store duplicate values.
- `multiset.find(x)` returns a pointer to the *first* x instead (who cares).

If you're special, you might want a **multimap**! (nobody ever wants this)

It can support duplicate keys! Which is excellent because it removes the only practical benefit of having a map (if you ask for the value at a key, it'll give you the value at that key).

Outline

1 Housekeeping

2 Ranges and pointers

3 What is a data structure?

4 A range sum thing?

5 Never heard of a linked list

6 AVL trees in practice

7 COMBO

8 Appendix and resources

Photographs (LC239)

Problem

A horizon line can be divided into N segments. The i'th segment has beauty b_i. Your camera can take a picture of K contiguous segments along the horizon, so there are $N - K + 1$ distinct photos you can take.

The beauty of a photograph is the **maximum** beauty of any segment in it. For each photograph $[i, i + K - 1]$ for $i \in \{1, ..., N - K + 1\}$, determine its beauty.

Part I: Try solve this in $O(N^2)$.

Part II: Try solve this in $O(N \log N)$.

Part III: Try solve this in $O(N)$.

Outline

1 Housekeeping

2 Ranges and pointers

3 What is a data structure?

4 A range sum thing?

5 Never heard of a linked list

6 AVL trees in practice

7 COMBO

8 **Appendix and resources**

Upcoming Events

Look out for our **IMC Coding Competition** tomorrow 3pm-6pm AGM G07!

Feedback!

Please fill out feedback form so we know how to run future workshops :D

(a) Feedback form

(b) Attendance form

Arc CLUBS SUPPORTED BY Arc INDEPENDENTLY RUN

UNSW CPMSoc